

JAVA INTERVIEW PREPARATION

CHAPTER 40

SPRING SECURITY:
TRUST, IDENTITY, AND POLICY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring Security: Filter Chains, Authentication, Authorization, CSRF, and Token Safety

Senior / Lead Java Developer Reading Chapter

Spring Security protects Servlet applications through an ordered filter chain. Authentication establishes who the caller is; authorization decides whether that authenticated identity may perform an action; exploit protections address attacks such as CSRF, session fixation, malicious headers, and unsafe request handling. Security is a system property spanning proxy, application, identity provider, browser, and data layer.

The senior-level mental model is not "add a login filter." It is a sequence of trust decisions. Where did the credential originate? Which component verified it? What exact identity and assurance were established? Which policy authorizes this action on this object? Where can untrusted input cross a tenant or privilege boundary? Spring Security supplies mechanisms, but the application still owns those answers.

Security also needs defense in depth. A route rule can prevent most unauthorized calls, method security can protect alternative invocation paths, repository predicates can enforce tenant isolation, and database privileges can limit blast radius. Repeating the same weak check at several layers is not defense in depth; each layer should protect a distinct boundary.

The Servlet Security Architecture

```
client
-> servlet container
-> DelegatingFilterProxy
-> FilterChainProxy
-> matching SecurityFilterChain
-> ordered security filters
-> DispatcherServlet and controller
```

FilterChainProxy selects the first SecurityFilterChain whose request matcher applies. Ordering matters. A broad chain placed first can shadow a more specific actuator or API chain.

```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/catalog/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
        .build();
}
```

Prefer explicit allow rules followed by an authenticated or deny-all default.

[DelegatingFilterProxy](#) is the servlet container bridge to a Spring-managed bean, usually [springSecurityFilterChain](#). That bean is a [FilterChainProxy](#), which evaluates configured chains and delegates to the first one that matches. The selected chain contains security filters in a framework-defined order so exploit protection, authentication, exception translation, and authorization occur at the correct stages.

This architecture has two ordering problems. Chain order decides which chain handles a request; filter order decides how controls compose inside that chain. A custom JWT filter inserted at an arbitrary position might run twice on error or async dispatch, run after authorization, or bypass Spring's exception handling. Prefer

built-in authentication support. When a custom filter is genuinely required, choose its position relative to a documented security filter and test every dispatch type.

```
request
-> first matching SecurityFilterChain only
    -> context loading
    -> exploit protections
    -> authentication mechanisms
    -> exception translation
    -> authorization
-> DispatcherServlet
```

With multiple chains, give each a deliberate matcher and an explicit order. A management chain using mutual TLS, an API chain using bearer tokens, and a browser chain using sessions can coexist, but a broad `/**` chain must remain last. End each policy with a clear default; unmatched endpoints should not become public because a developer forgot a rule.

Authentication Objects

`SecurityContextHolder` stores the current `SecurityContext`, which contains `Authentication`. `Authentication` holds principal, credentials where relevant, authorities, details, and authentication state.

```
credentials/token -> authentication filter
-> AuthenticationManager
-> AuthenticationProvider
-> authenticated Authentication
-> SecurityContext
```

`ProviderManager` delegates to `AuthenticationProvider` implementations. Providers can validate passwords, tokens, certificates, or custom credentials.

Do not treat `authentication.isAuthenticated()` alone as proof of a specific assurance level. Inspect trusted token validation, authorities, claims, and authentication mechanism.

Before authentication, an `Authentication` object can represent a submitted credential. An authentication provider verifies it and returns a trusted object whose principal and authorities are suitable for later decisions. Credentials should be erased after verification where the mechanism supports it. Application code should avoid retaining the original password or bearer token in domain objects, logs, sessions, or exception details.

`ProviderManager` tries compatible providers rather than sending every credential to every provider. A username/password token may be handled by a DAO provider, while a bearer token is handled by resource-server support. Provider ordering and failure behavior matter when several mechanisms overlap. An anonymous authentication object can also exist; therefore checking for non-null authentication is not equivalent to requiring a logged-in user.

Authentication establishes an identity under a particular mechanism. It does not prove that every claim is suitable for authorization. Map external groups, scopes, or roles into application authorities at one controlled boundary. Reject malformed or ambiguous claims instead of allowing each controller to interpret them differently.

Security Context Lifecycle

In common Servlet execution, the context is associated with the current thread for request processing and cleared by security infrastructure afterward.

Starting arbitrary executor work does not automatically propagate it. Use `DelegatingSecurityContext` executors or supported context-propagation mechanisms when background work should run as a caller. Often the safer design is to pass an immutable user or tenant identifier and re-authorize at the new boundary.

Never reuse one user's context across pooled tasks. Clear context in custom integrations.

Thread-local storage is convenient during a synchronous servlet request, but executor pools reuse threads. Copying a context without disciplined setup and cleanup can authorize work as the previous user.

`DelegatingSecurityContextExecutor` and related wrappers capture or establish the intended context and clear it afterward, but context propagation should not become invisible authorization.

For durable background work, pass a stable actor and tenant identity as audited data, not a live `SecurityContext`. When a message is consumed later, establish a service identity and re-evaluate the permissions appropriate to that boundary. A request-time authorization decision is not automatically valid hours later after roles or resource ownership change.

Authorization

Request authorization protects routes; method security protects service invocations.

```
@EnableMethodSecurity
@Configuration
class MethodSecurityConfiguration { }

@PreAuthorize("hasAuthority('order:write') and @ownership.canEdit(#id, authentication)")
public void cancel(OrderId id) { }
```

Method security is valuable when the service can be called from controllers, messages, schedules, or other beans. It is proxy-based, so self-invocation and objects outside the container matter.

Keep authorization policy understandable. Complex SpEL that queries several services can hide I/O and create inconsistent decisions. A dedicated authorization service or policy object can be clearer.

Request rules answer whether this HTTP exchange may reach an endpoint. Method rules answer whether this invocation may execute, regardless of whether it came from MVC, messaging, a scheduler, or another bean. Object authorization answers whether the caller may act on this particular order or account. Mature systems usually need all three.

Method security is interceptor/proxy based. A call from one method to another on the same instance can bypass the proxy, and an object created with `new` is not protected by Spring interceptors. Keep secured operations on clear service boundaries and test invocation paths rather than assuming the annotation is magic.

Policy should be fail closed and deterministic under partial failure. If an external policy service is unavailable, silently permitting access is rarely acceptable. If business requirements demand cached authorization or emergency fallback, define staleness, revocation, audit, and blast radius explicitly.

Roles and Authorities

Authorities are strings representing permissions or scopes. Role helpers commonly add a `ROLE_` prefix by convention.

```
hasRole("ADMIN")           commonly checks authority ROLE_ADMIN
hasAuthority("ADMIN")      checks exactly ADMIN
```

Avoid accidental prefix mismatches. Prefer granular authorities such as `order:read` and `order:cancel`, then map identity-provider roles to application permissions at a controlled boundary.

Roles describe organizational grouping; authorities describe capabilities the application evaluates. Directly embedding identity-provider group names throughout code couples policy to an external directory. A claim conversion layer can map `warehouse-supervisors` to `order:cancel` today and change that mapping later

without rewriting every annotation.

Avoid an authority for every individual object. Thousands of `order:123:read` strings inflate tokens and become stale. Use coarse capability authorities plus object-level ownership or tenant policy based on current data.

Password Storage

Passwords must be stored using an adaptive one-way password encoder such as bcrypt, PBKDF2, scrypt, or Argon2 with parameters appropriate to current operational guidance.

```
PasswordEncoder encoder = PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

Delegating format identifiers support algorithm migration. Tune verification cost against hardware and authentication load. Rate limiting and breached-password controls complement hashing; they do not replace it.

Never log passwords, password hashes, authorization headers, refresh tokens, or client secrets.

Adaptive hashes are intentionally expensive and salted. Their work factor should be tuned on production-like hardware so verification resists guessing without exhausting the authentication service. Because hardware improves, encoded hashes need an algorithm and parameter identifier so successful login can trigger migration to a stronger setting.

Hashing cannot compensate for weak enrollment and recovery. Apply rate limits, multifactor authentication where risk demands it, secure password reset tokens, breached-password screening, and session revocation after account recovery. A fast general-purpose hash such as SHA-256 is not a password storage algorithm, even if a salt is added manually.

Session Versus Token Authentication

Server sessions store authentication state under a session identifier, normally delivered in a secure cookie. Bearer-token resource servers validate presented tokens for each request.

```
session: cookie -> server/session store -> SecurityContext  
JWT:      bearer token -> signature/claims validation -> Authentication
```

JWTs are signed, not automatically encrypted. Claims are visible to the holder. Validate signature algorithm, issuer, audience, timestamps, and required claims. Key rotation and clock skew need policy.

Revocation is harder for self-contained access tokens. Use short access-token lifetime, refresh-token controls, introspection, deny lists where justified, or identity-provider revocation semantics.

Sessions and tokens move state; they do not eliminate it. A session keeps most authentication state server-side and sends an opaque identifier. A self-contained token carries claims to each resource server, reducing lookup cost but making stale claims and emergency revocation harder. The correct choice follows client type, deployment boundary, revocation requirement, and identity architecture.

JWT validation is more than signature verification. Pin allowed algorithms, select keys from a trusted issuer configuration, validate issuer and intended audience, check expiry and not-before with bounded clock skew, and require claims used by policy. Never accept the algorithm declared by an untrusted token without an allowlist. A token valid for service A must not be accepted by service B merely because both trust the same signing key.

Access tokens should contain only what resource servers need. They are commonly base64url-encoded and readable by the holder, so secrets and unnecessary personal data do not belong in claims. Keep

access-token lifetimes short and protect refresh tokens as high-value credentials, often with rotation and reuse detection.

CSRF

CSRF exploits a browser automatically attaching credentials such as cookies to a malicious cross-site request. Spring Security enables CSRF protection for appropriate browser flows by default.

Disabling CSRF is not justified merely because an endpoint returns JSON. Ask how credentials are sent. A stateless API using an Authorization bearer header not automatically attached by the browser may not need CSRF tokens, while cookie-authenticated APIs do.

```
browser cookie automatically sent -> CSRF risk
explicit bearer header by API client -> different threat model
```

CORS and CSRF solve different problems. CORS controls which browser origins can read or make selected cross-origin requests; it is not authentication and does not prevent all request submission.

The attack works because the browser supplies ambient credentials without the victim application choosing to do so. A malicious page submits a state-changing request; the target browser attaches the victim's session cookie; without a second value unavailable to the attacker, the server cannot distinguish the submission from the legitimate site.

```
evil.example form
-> POST bank.example/transfer
    browser automatically adds bank session cookie
    server sees authenticated request
```

```
CSRF token defense:
legitimate page obtains unpredictable token
unsafe request sends cookie + matching token
attacker cannot read or predict the required token
```

SameSite cookies reduce risk but do not replace a complete threat analysis. Browser behavior, legacy clients, sibling subdomains, and login/logout flows still matter. Likewise, putting a JWT in a cookie recreates ambient credential behavior; the fact that the credential format is JWT does not remove CSRF.

CORS

Define exact origins, methods, headers, and credential policy. `Access-Control-Allow-Origin: *` cannot safely combine with credentials and is rarely appropriate for private APIs.

Preflight OPTIONS requests must reach CORS handling before authorization blocks them incorrectly. Coordinate reverse-proxy and application CORS behavior rather than adding contradictory headers in both.

CORS is enforced by browsers. It does not stop curl, server-to-server clients, or an attacker from sending a request. It controls whether browser JavaScript from one origin may access the response under the declared method, header, and credential rules. Therefore it cannot replace authentication or object authorization.

When credentials are allowed, echo only reviewed origins and include the correct `Vary: Origin` behavior so shared caches do not mix policies. Origin matching should parse and compare origins, not use unsafe substring checks such as `"endsWith example.com"` that accidentally trust `evil-example.com`.

Security Headers and Sessions

Spring Security can write protective headers such as cache, content-type, frame, and transport policies according to configuration. Content Security Policy requires application-specific directives and is not safely guessed by a framework.

For sessions, use `Secure`, `HttpOnly`, and appropriate `SameSite` cookies, rotate identifiers after authentication, define inactivity and absolute timeout, and consider distributed session storage only when architecture requires it.

`Secure` restricts cookie transmission to secure transport, `HttpOnly` prevents normal JavaScript access, and `SameSite` controls cross-site inclusion. These flags address different threats and should be verified at the external endpoint after proxies and TLS termination. Trust forwarded headers only from known proxies so an attacker cannot influence scheme or secure-cookie decisions.

Session fixation protection rotates the session identifier when authentication changes. Logout should invalidate server state and clear cookies according to the chosen mechanism. In a cluster, distributed sessions improve availability but introduce serialization, latency, and cleanup concerns; statelessness should not be pursued merely to avoid understanding those tradeoffs.

Security headers such as content-type options and frame protections have useful defaults. Content Security Policy is application-specific because it describes permitted script, style, image, and connection sources. Roll it out with reporting and remove unsafe allowances rather than copying a policy that breaks the application or permits everything.

OAuth 2.0 and OIDC Boundaries

OAuth 2.0 is an authorization framework; OpenID Connect adds identity. A resource server validates access tokens. An OAuth client initiates authorization flows and obtains tokens. An authorization server issues tokens.

Do not implement a custom authorization server or token parser casually. Use supported Spring Security components and a maintained identity provider. OAuth grant choice depends on client type and trust boundary; modern browser and native clients commonly use authorization code with PKCE.

An ID token tells an OAuth client about authentication at the authorization server; an access token authorizes calls to a resource server. Sending an ID token to an API in place of an access token confuses audiences and token purposes. The resource server must validate the token intended for it and derive authorities through an explicit claim converter.

PKCE binds an authorization request to the client instance that later exchanges the code, reducing interception risk for public clients that cannot hold a client secret. `state` protects the client flow against request forgery and response mix-up; OIDC `nonce` binds the authentication response. These values serve different purposes.

Key rotation must be an operational design, not an afterthought. Resource servers cache discovered keys, tolerate overlap during rotation, and fail safely when refresh fails. Monitor unknown key IDs and issuer connectivity without logging tokens.

Multi-Tenancy and Object Authorization

Route-level authority is insufficient when access depends on object ownership or tenant.

```
authenticated tenant T1 requests /orders/07
database query must constrain tenant_id = T1 and order_id = 07
```

Do not load an object globally and only afterward check tenant if the error or timing can leak existence. Push tenant constraints into repository queries and retain database defense where possible.

Never trust tenant ID supplied in a header without binding it to authenticated claims and authorization.

The safest query makes unauthorized data unselectable:

```
Optional<Order> findByIdAndTenantId(UUID id, UUID tenantId);
```

Loading by ID and checking tenant afterward can leak existence through timing, caching, logs, or different error messages. Repository scoping also reduces the damage of a missed controller check. For especially sensitive systems, database row-level security can add another independent boundary, but it requires reliable tenant context on every connection and careful pool cleanup.

Administrative cross-tenant access should be an explicit capability with strong audit, not a special tenant ID or a boolean hidden in a request header. Record actor, target tenant, reason, and policy decision according to compliance requirements.

Error Handling

AuthenticationEntryPoint handles missing or invalid authentication. AccessDeniedHandler handles forbidden access for an authenticated caller. Return consistent problem bodies without exposing token-validation internals.

401 normally invites authentication and may include `WWW-Authenticate`; 403 means the caller is authenticated but not allowed. Deliberate 404 concealment can be valid for protected resources.

Do not return the token decoder's raw message. Details such as key IDs, expected issuer, claim parsing, or directory state help attackers and produce an unstable contract. Return a bounded problem code while retaining the internal cause in secure telemetry.

Authentication and authorization failures are expected security outcomes, not application defects, but spikes can indicate attack or configuration breakage. Measure them by bounded reason and route template. Apply rate limits carefully so an attacker cannot use one victim identifier to deny service to that victim.

Testing Security

Test the real filter chain, CSRF behavior, method security, role/authority mapping, token validation, CORS, and error bodies.

```
anonymous -> public and protected routes
authenticated without authority -> denied
authenticated with authority -> permitted
wrong tenant/object -> denied or concealed
expired/wrong issuer/wrong audience token -> rejected
cookie state change without CSRF token -> rejected
```

Mocked authentication shortcuts are useful but do not prove decoder, filter order, or claim conversion.

Use three layers. Policy unit tests exercise ownership and tenant decisions without Spring. MVC security tests verify request matchers, CSRF, entry points, and method rules. Integration tests use a real or standards-conformant token issuer setup to prove algorithm, issuer, audience, expiry, and authority conversion. Negative tests are more valuable than a single happy path.

Test chain shadowing explicitly when multiple `SecurityFilterChain` beans exist. Verify actuator, error dispatches, static resources, documentation, and unknown endpoints. Also test that an authenticated user with the correct coarse permission cannot cross an object or tenant boundary.

Production Perspective

Log security outcomes without credentials: authentication mechanism, bounded failure reason, route template, policy decision, and correlation ID. Prevent username or token values from becoming high-cardinality metrics.

Protect Actuator, documentation, error, static, and management routes explicitly. Monitor authentication failures, access denials, key-fetch errors, token expiry patterns, session growth, and authorization latency. Fail closed when policy infrastructure is unavailable unless an explicitly reviewed fallback says otherwise.

Configuration itself needs review and drift detection. A temporary `permitAll`, disabled CSRF, wildcard origin, or debug endpoint can survive into production if security is verified only through code inspection. Automated tests should assert the effective routes, and deployment checks should inspect external headers, cookies, TLS, and proxy behavior.

Incident readiness includes signing-key rotation, client-secret replacement, session invalidation, refresh-token revocation, and audit search. A design that can validate credentials but cannot revoke or investigate them is incomplete. Keep sensitive audit access restricted and define retention before an incident occurs.

Common Senior-Level Traps

The most common trap is mixing authentication with authorization: a valid token proves only that a trusted issuer made statements about a subject, not that the subject may access this order. Another is disabling CSRF because the API returns JSON while continuing to authenticate with cookies. A third is trusting a tenant header independently from the authenticated identity.

Other traps include accepting any token signed by a familiar key without audience validation, placing external group names directly in business code, using wildcard CORS with credentials, adding a custom filter in the wrong order, and assuming method security covers self-invocation. Security failures often occur in composition rather than in one obviously broken line.

A Strong Interview Answer

I model Spring Security as an ordered trust pipeline. FilterChainProxy selects the first matching SecurityFilterChain, authentication mechanisms validate credentials and create a trusted Authentication, and request plus method policy authorize capabilities. Object and tenant authorization are constrained again in data access. I prefer supported session or resource-server components over custom filters, validate JWT algorithm, issuer, audience, time, and required claims, and map external roles at one boundary. CSRF follows ambient browser credentials, so cookie-authenticated JSON still needs protection; CORS is only a browser cross-origin policy. I test negative paths through the real chain, secure management and error routes, propagate context deliberately, and design revocation, rotation, audit, and failure-closed behavior for production.

Interview-Ready Summary

Spring Security's Servlet model uses FilterChainProxy and ordered SecurityFilterChains. Authentication produces an Authentication in a SecurityContext; authorization evaluates requests and methods. Method security is proxy-based and object-level policy must include ownership and tenant constraints.

CSRF depends on automatically attached browser credentials, not response format. CORS is not authentication. Validate tokens completely, hash passwords adaptively, propagate context only through supported mechanisms, and secure error and management paths. Test the actual filter chain and failure cases.

Revision Notes

- FilterChainProxy delegates to the first matching SecurityFilterChain.
- Filter order places authentication before authorization.
- SecurityContext contains the current Authentication.
- AuthenticationManager delegates to AuthenticationProvider implementations.
- Request and method authorization protect different call paths.
- Method security is proxy-based and subject to self-invocation limits.
- Roles commonly use a ROLE_ prefix; authorities are exact strings.
- Use adaptive one-way password hashing.
- JWTs are signed but not automatically encrypted.
- Validate token issuer, audience, algorithm, time, and required claims.
- Short token lifetime and refresh policy address revocation tradeoffs.
- CSRF concerns browser-automatically-attached credentials.
- JSON APIs using cookies can still be vulnerable to CSRF.
- CORS is distinct from authentication and CSRF.
- Configure exact credentialed origins rather than wildcard access.
- Security context does not automatically cross arbitrary executors.
- Bind tenant identity to authenticated claims.
- Enforce object ownership in data access and policy.
- AuthenticationEntryPoint and AccessDeniedHandler own security failures.
- Test real filters, tokens, CSRF, CORS, method policy, and tenant isolation.